

Mid-Term Examination

ACOL 216: Introduction to Computer Architecture

IIT Delhi - Abu Dhabi · Semester II, 2025-26

Total Marks: 25

Time: 50 minutes

1. Prove each of the following Boolean expressions using Boolean algebraic laws.

(i) $(A' + B') \cdot (A' + B) \cdot (A + B' \cdot D + C) = A' \cdot B' \cdot D + A' \cdot C$

(ii) $A \cdot B' + A' \cdot B' + A \cdot B' \cdot C \cdot D = B'$

(2.5 × 2 = 5 marks)

(i) **Proof for** $(A' + B') \cdot (A' + B) \cdot (A + B' \cdot D + C) = A' \cdot B' \cdot D + A' \cdot C$

$$\begin{aligned} \text{LHS} &= (A' + B') \cdot (A' + B) \cdot (A + B' \cdot D + C) \\ &= (A' + B' \cdot B) \cdot (A + B' \cdot D + C) && \text{(Distributive Law)} \\ &= (A' + 0) \cdot (A + B' \cdot D + C) && \text{(Complement Law: } B' \cdot B = 0) \\ &= A' \cdot (A + B' \cdot D + C) && \text{(Identity Law: } A' + 0 = A') \\ &= A' \cdot A + A' \cdot B' \cdot D + A' \cdot C && \text{(Distributive Law)} \\ &= 0 + A' \cdot B' \cdot D + A' \cdot C && \text{(Complement Law: } A' \cdot A = 0) \\ &= A' \cdot B' \cdot D + A' \cdot C && \text{(Identity Law)} \\ &= \text{RHS} \end{aligned}$$

(ii) **Proof for** $A \cdot B' + A' \cdot B' + A \cdot B' \cdot C \cdot D = B'$

$$\begin{aligned} \text{LHS} &= A \cdot B' + A' \cdot B' + A \cdot B' \cdot C \cdot D \\ &= B' \cdot (A + A') + A \cdot B' \cdot C \cdot D && \text{(Distributive Law)} \\ &= B' \cdot (1) + A \cdot B' \cdot C \cdot D && \text{(Complement Law: } A + A' = 1) \\ &= B' + A \cdot B' \cdot C \cdot D && \text{(Identity Law)} \\ &= B' \cdot (1 + A \cdot C \cdot D) && \text{(Distributive Law)} \\ &= B' \cdot (1) && \text{(Null/Annihilator Law: } 1 + X = 1) \\ &= B' && \text{(Identity Law)} \\ &= \text{RHS} \end{aligned}$$

2. A number is known as a cubic Armstrong number if the sum of the cubes of its individual decimal digits is equal to the number itself. For example, 153 is a cubic Armstrong number ($153 = 1^3 + 5^3 + 3^3$).

You are given a number in register r_0 , and it is known to be between 1 and 1,000,000. **Write a piece of assembly code in SimpleRisc to determine if this number is a cubic Armstrong number.** Save 1 in register r_1 if it is a cubic Armstrong number; otherwise, save 0. (5 marks)

Solution:

```
mov r1, 0      @ Initialize r1 to 0 (default: not an Armstrong number)
mov r2, 0      @ r2 will act as the accumulator for the sum of cubes
mov r3, r0     @ Copy the original number to r3 for processing
mov r6, 10     @ Constant 10 used for modulo and division
```

.loop:

```
    cmp r3, 0      @ Check if the working number has been reduced to 0
    beq .evaluate  @ If yes, all digits are processed; exit loop

    mod r4, r3, r6  @ r4 = r3 % 10 (Extract the least significant digit)
    div r3, r3, r6  @ r3 = r3 / 10 (Discard the extracted digit)

    mul r5, r4, r4  @ r5 = r4 * r4 (digit squared)
```

```

mul r5, r5, r4    @ r5 = r5 * r4 (digit cubed)

add r2, r2, r5    @ Add the cubed digit to the running sum
b .loop          @ Branch back to start of loop

.evaluate:
  cmp r0, r2      @ Compare original number (r0) with the sum of cubes (r2)
  beq .true       @ If equal, branch to set success
  b .done         @ Else, skip to end (r1 is already 0)

.true:
  mov r1, 1       @ Set r1 to 1 (It is a cubic Armstrong number)

.done:
  @ Execution completes here. The boolean result is successfully stored in r1.

```

3. Draw the circuit diagram of a standard 4-bit ripple-carry adder. Then, explain how you can configure or augment this exact same adder circuit (without fundamentally changing its internal structure) to perform the following specific operations on a 4-bit input A :

- (i) 2's Complement Subtraction: Computing $A - B$ (where B is a second 4-bit input).
- (ii) Increment: Computing $A + 1$.
- (iii) Decrement: Computing $A - 1$.

(2+1+1+1 = 5 marks)

Solution:

Let the standard 4-bit ripple-carry adder have two 4-bit inputs, A and X , and a 1-bit carry-in, C_{in} . The basic function of this adder is to compute the sum $S = A + X + C_{in}$. We can configure or augment this exact same circuit to perform the requested operations by manipulating the second input (X) and the carry-in (C_{in}).

(i) **2's Complement Subtraction ($A - B$):**

To subtract B from A , we add the 2's complement of B to A . The 2's complement of B is obtained by taking the 1's complement (inverting the bits, denoted as B') and adding 1.

- **Input A :** A
- **Input X :** B' (Invert all bits of B , typically done by passing B through NOT gates or XORing B with 1)
- **Carry-in (C_{in}):** 1

Expression: $S = A + B' + 1$

(ii) **Increment ($A + 1$):**

To increment A , we need to add 1 to the operand. This is achieved by hardwiring the second input to zero and forcing the carry-in to 1.

- **Input A :** A
- **Input X :** 0000_2 (All bits connected to logic LOW / Ground)
- **Carry-in (C_{in}):** 1

Expression: $S = A + 0000_2 + 1$

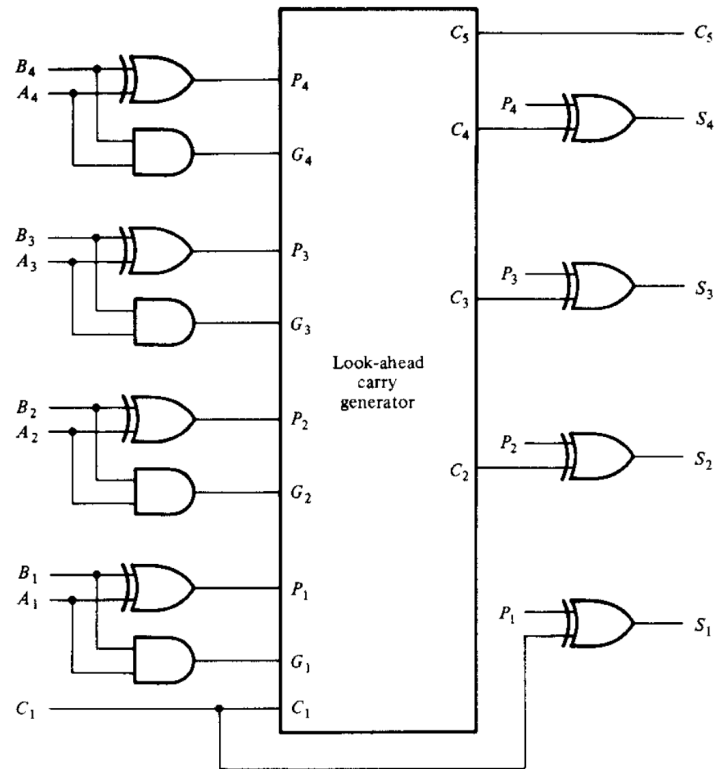
(iii) **Decrement ($A - 1$):**

To decrement A , we must add -1 to A . In a 4-bit 2's complement system, -1 is represented by the binary value 1111_2 .

- **Input A :** A
- **Input X :** 1111_2 (All bits connected to logic HIGH / V_{cc})
- **Carry-in (C_{in}):** 0

Expression: $S = A + 1111_2 + 0$

4. The figure below shows the schematic of a 4-bit look-ahead carry generator. The inputs are $A_1 \dots A_4$ and $B_1 \dots B_4$, with an initial carry-in of C_1 .



Derive the two-level Boolean expression for the final output carry C_5 strictly in terms of the generate variables ($G_1 \dots G_4$), propagate variables ($P_1 \dots P_4$), and the initial carry C_1 . You must show the step-by-step unrolling of the equations to fully expand the final expression. **(5 marks)**

Solution:

In a carry look-ahead adder, the carry-out for any given stage i is defined by the fundamental equation:

$$C_{i+1} = G_i + P_i \cdot C_i$$

To derive the two-level Boolean expression for the final carry C_5 strictly in terms of G , P , and C_1 , we must step-by-step unroll the equations starting from C_2 :

Step 1 (Express C_2):

$$C_2 = G_1 + P_1 \cdot C_1$$

Step 2 (Express C_3 and substitute C_2):

$$\begin{aligned} C_3 &= G_2 + P_2 \cdot C_2 \\ &= G_2 + P_2 \cdot (G_1 + P_1 \cdot C_1) \\ &= G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot C_1 \end{aligned}$$

Step 3 (Express C_4 and substitute C_3):

$$\begin{aligned} C_4 &= G_3 + P_3 \cdot C_3 \\ &= G_3 + P_3 \cdot (G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot C_1) \\ &= G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot C_1 \end{aligned}$$

Step 4 (Express C_5 and substitute C_4):

$$\begin{aligned} C_5 &= G_4 + P_4 \cdot C_4 \\ &= G_4 + P_4 \cdot (G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot C_1) \\ &= G_4 + P_4 \cdot G_3 + P_4 \cdot P_3 \cdot G_2 + P_4 \cdot P_3 \cdot P_2 \cdot G_1 + P_4 \cdot P_3 \cdot P_2 \cdot P_1 \cdot C_1 \end{aligned}$$

Final Two-Level Expression:

$$C_5 = G_4 + P_4 G_3 + P_4 P_3 G_2 + P_4 P_3 P_2 G_1 + P_4 P_3 P_2 P_1 C_1$$

5. (a) State the fundamental physical components used to store a single bit of information in a 6-transistor SRAM cell versus a DRAM cell.
- (b) Explain why a DRAM array requires *periodic refreshing* while an SRAM array does not. What specific steps occur during a DRAM refresh operation?
- (c) In an array of SRAM cells, reading is optimized using a technique called *pre-charging*. Briefly explain the physical issue this technique solves and how it achieves faster read times.

(1+2+2 = 5 marks)

Solution:

(a) **Physical Components:**

- **SRAM cell:** Stores a single bit using **6 Transistors** (typically arranged as two cross-coupled inverters that hold the state, plus two access transistors).
- **DRAM cell:** Stores a single bit using **1 Capacitor and 1 Transistor** (1T1C). The capacitor holds the electrical charge representing the bit, and the transistor acts as an access switch.

(b) **DRAM Refreshing:**

DRAM requires periodic refreshing because its storage capacitors naturally leak charge over time due to leakage currents in the access transistor and the dielectric. If not restored, the stored voltage levels decay, resulting in data loss. SRAM does not require refreshing because its cross-coupled inverters are actively connected to the power supply (V_{DD}) and ground, continuously driving the logic state as long as power is applied.

Specific steps during a refresh operation:

- (a) **Activation:** A specific row (wordline) is activated.
- (b) **Charge Sharing:** The capacitors in that row dump their charge onto the bitlines, slightly altering the bitline voltage.
- (c) **Sensing & Amplification:** Sense amplifiers detect these minute voltage changes and pull the lines to full logic levels (V_{DD} or $0V$).
- (d) **Write-back (Restore):** The fully amplified logic levels are driven back into the capacitors of the activated row, completely restoring their stored charge.
- (c) **SRAM Pre-charging:**
Physical Issue: The bitlines (BL and BL') in an SRAM array are long metal wires shared by many cells, giving them a very large parasitic capacitance. A single, small SRAM cell transistor would take an impractically long time to fully discharge this massive capacitance from V_{DD} to $0V$ during a read.

How it achieves faster read times: Before the read occurs, the pre-charge circuit pulls both BL and BL' to the same high voltage level (typically V_{DD}). When the wordline is turned on, the SRAM cell only needs to pull down one of the bitlines very slightly. A highly sensitive *sense amplifier* then detects this tiny voltage differential (ΔV) between BL and BL' and rapidly amplifies it to a full logic level output. Because the cell only has to create a small voltage drop rather than swinging the entire bitline to ground, the read speed is vastly accelerated.